

Local Pulse

Software Requirements Specification

INT 222

ADVANCED WEB DEVELOPMENT

Student Names

Anand Singh

Saran Teja Gandhi

Aditya Gautam

Student Registration Numbers.

Anand Singh:12301133

Saran Teja Gandhi:123120130

Aditya Gautam:12419605

Prepared for
Continuous Assessment 3
Spring 2025

Table of Contents

REVISION HISTORY	
1. INTRODUCTION	3
1.1 PURPOSE	3
1.2 SCOPE	3
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS	3
1.4 REFERENCES	4
1.5 OVERVIEW	4
2. GENERAL DESCRIPTION	4
2.1 PRODUCT PERSPECTIVE	4
2.2 PRODUCT FUNCTIONS	4
2.3 USER CHARACTERISTICS.....	4
2.4 GENERAL CONSTRAINTS.....	5
2.5 ASSUMPTIONS AND DEPENDENCIES	4
3. SPECIFIC REQUIREMENTS.....	6
3.1 EXTERNAL INTERFACE REQUIREMENTS	6
3.1.1 <i>User Interfaces</i>	6
3.1.2 <i>Hardware Interfaces</i>	7
3.1.3 <i>Software Interfaces</i>	7
3.1.4 <i>Communications Interfaces</i>	7
3.2 FUNCTIONAL REQUIREMENTS	6
3.2.1 <i><Functional Requirement or Feature #1></i>	6
3.2.2 <i><Functional Requirement or Feature #2></i>	7
3.2.3 <i><Functional Requirement or Feature #></i>	7
3.2.4 <i><Functional Requirement or Feature #4></i>	8
3.2.5 <i><Functional Requirement or Feature #5></i>	8
3.2.6 <i><Functional Requirement or Feature #6></i>	9
3.2.7 <i><Functional Requirement or Feature #7></i>	10
3.5 NON-FUNCTIONAL REQUIREMENTS	11
3.5.1 <i>Performance</i>	11
3.5.2 <i>Reliability</i>	12
3.5.3 <i>Availability</i>	12
3.5.4 <i>Security</i>	13
3.5.5 <i>Maintainability</i>	13
3.5.6 <i>Portability</i>	14
3.7 DESIGN CONSTRAINTS	14
3.9 OTHER REQUIREMENTS	15
4. ANALYSIS MODELS.....	15
4.1 DATA FLOW DIAGRAMS (DFD)	15
5. GITHUB LINK.....	16
6. DEPLOYED LINK.....	16
A. APPENDICES.....	17
A.1 APPENDIX 1	17
A.2 APPENDIX 2.....	17

1. Introduction

The Local Business Directory and Product Finder is a web platform that helps customers find products in nearby local stores. Many small businesses do not have strong online visibility, so customers cannot easily know which store has the product they need. This platform allows businesses to register, create a profile, and add product listings, while customers can search products and view the business details and contact information.

1.1 Purpose

This document is prepared to define the software requirements for the **Local Business Directory and Product Finder** application. It acts as a reference for designing, developing, and testing the system. It also ensures that all project members and reviewers have the same understanding of the system features and expected behavior.

1.2 Scope

Local businesses often don't have strong online visibility for customers searching for a specific product nearby. Customers usually call multiple stores or travel without confirmation.

This product will:

- allow customers to **search products** (example: "Notebook", "iPhone charger")
- show **nearby businesses** that sell the product
- provide a **directory** of businesses with categories and contact details
- allow business owners to **add and manage** their product listings

In Scope

- Customer search + view business/product details
- Business owner portal (profile + product CRUD)
- Filters (category, price range, distance/location)
- Basic contact options (call/email/website/directions link)

Out of Scope (for now)

- Payments / online ordering
- Delivery tracking
- Real-time inventory sync with POS systems

1.3 Definitions, Acronyms, and Abbreviations

- UI: User Interface
- API: Application Programming Interface
- DB: Database
- CRUD: Create, Read, Update, Delete
- RLS: Row Level Security (Supabase security feature)
- Supabase: Cloud service that provides PostgreSQL DB + Auth + Storage + APIs

1.4 References

- React + TypeScript documentation
- Node.js and Express documentation
- Supabase documentation (PostgreSQL, Auth, RLS, Storage)

1.5 Overview

- Section 2 explains the product, users, constraints, and assumptions.
- Section 3 lists all functional and non-functional requirements.
- Section 4 provides analysis models (DFD).
- Sections 5–6 contain deployment/repository links.

2. General Description

2.1 Product Perspective

This is a standalone web application with a client-server architecture:

- Frontend: React + TypeScript (web UI)
- Backend: Node.js + Express (REST API)
- Database: Supabase PostgreSQL (data storage)
Optional: Supabase Auth + Storage for login and images.

2.2 Product Functions

Main functions include:

- Signup and login (Customer / Business Owner)

- Business profile creation and editing
- Product listing management (add/edit/delete)
- Product search by keyword and filters
- Business directory browsing (categories)
- Business details page with products
- Contact options for customers (phone/email/website)

2.3 User Characteristics

Main functions include:

- Signup and login (Customer / Business Owner)
- Business profile creation and editing
- Product listing management (add/edit/delete)
- Product search by keyword and filters
- Business directory browsing (categories)
- Business details page with products
- Contact options for customers (phone/email/website)

2.4 General Constraints

- Must support latest versions of Chrome/Edge/Firefox
- Must support mobile view (responsive design)
- Must store data in PostgreSQL via Supabase
- Must protect business-only actions using authentication + role checks
- Search should be fast and handle large data using pagination

2.5 Assumptions and Dependencies

- Users have internet access
- Businesses provide correct product info (name, price, availability)
- Location can be:
customer-entered (city/zip), OR
browser location permission (optional)

- Product images depend on storage availability (Supabase storage or similar)

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

Customer UI Pages

1. Home/Search

- Search bar (product name)
- Location input (city/zip)
- Filters button (category, price range, distance)

2. Search Results

- List of products + business name + address + distance (if location available)
- Sorting (nearest, lowest price, most relevant)

3. Product Details

- Product name, price, description, availability
- Business card preview + link to business page

4. Business Details

- Business name, category, address, phone, hours, map link
- Product list of that business

5. Login/Signup

- Choose role (Customer / Business Owner)
- Email + password form

Business Owner UI Pages

1. Owner Dashboard

- Business status
- Manage products button
- Quick stats (optional: total products)

2. **Business Profile**

- Edit name, address, phone, hours, description, category, images/logo

3. **Product Management**

- Add product form
- Edit/delete product controls
- Toggle availability (in stock/out of stock)

3.1.2 **Hardware Interfaces**

Not required for core system.

Optional:

- Device GPS (only if user allows browser location access)

3.1.3 **Software Interfaces**

- **Backend REST API** (Node/Express)
- **Supabase PostgreSQL** tables for users, businesses, products, categories
- **Auth:**
- Supabase Auth OR JWT sessions
- **Storage (optional):**
- Supabase Storage for product/business images

3.1.4 **Communications Interfaces**

- Client ↔ Server communication uses **HTTPS**
- Data format is **JSON**
- Errors returned with proper status codes (400, 401, 403, 404, 500)

3.2 **Functional Requirements**

3.2.1 **User Registration/Login**

Introduction: Allow users to create accounts and access features based on role.

Inputs:

- Email
- Password
- Role (Customer / Business Owner)

Processing:

- Validate email format and password rules
- Create user record and role
- Authenticate and generate session/token
- Redirect based on role

Outputs:

- Successful login
- User session active (token/cookie)
- Redirect:
 - Customer → Home/Search
 - Business Owner → Dashboard

Error Handling:

- Email already exists
- Invalid email/password
- Wrong password
- Server/Auth service failure

3.2.2 Create Business Profile (Business Owner)

Introduction: A business owner creates a business profile to appear in the directory.

Inputs:

- Business name
- Category
- Address (and optional geo-coordinates)
- Phone number
- Store hours
- Description

- **Images/logo (optional)**

Processing:

- Check logged-in user is Business Owner
- Validate required fields
- Save profile in DB linked to owner_id
- (Optional) Convert address → lat/lng for distance search

Outputs:

- Business profile created
- Business visible in directory

Error Handling:

- Missing required fields
- Duplicate profile rules (optional)
- Unauthorized (wrong role)
- DB failure

3.2.3 Add Product Listing (Business Owner)

Introduction: Business owners add products so customers can find them.

Inputs:

- Product name
- Category
- Price
- Description
- Tags/keywords (optional)
- Availability (in stock/out of stock)
- Image (optional)

Processing:

- Verify owner permission for the business
- Validate product name and price
- Save product linked to business_id
- Index searchable fields (name/tags)

Outputs:

- Product listing created and visible in search

Error Handling:

- Invalid price (negative/empty)
- Missing product name
- Unauthorized (not the business owner)
- Upload/storage error (if image)

3.2.4 Edit/Delete Product Listing

Introduction: Owners can update product details and remove old products.

Inputs:

- product_id
- Updated fields (name/price/category/availability/etc.) OR delete request

Processing:

- Permission check (owner of that business)
- Update record OR delete record
- Return latest product data

Outputs:

- Updated listing OR confirmation of deletion

Error Handling:

- Product not found
- Unauthorized access
- DB failure

3.2.5 Search Products (Customer)

Introduction: Customers search products and view nearby businesses that sell them.

Inputs:

- Search query (text)
- Location input (city/zip) or browser location (optional)
- Filters:
 - Category

Local Pulse

- Price range
- Distance range
- Availability only (optional)
- Sorting option (nearest/lowest price/relevance)

Processing:

- Sanitize query text
- Search product table (name + tags)
- Join with business table for store details
- Apply filters + pagination
- Calculate distance if coordinates exist

Outputs:

- Paginated list of matching products with:
 - product name, price, availability
 - business name, address, phone/hours
 - distance (if available)

Error Handling:

- No results found → show “No results” message
- Missing location → allow search without distance
- Server/DB error

3.2.6 View Business Details

Introduction: Customer can open a business page to view details and its product list.

Inputs:

- business_id

Processing:

- Fetch business profile
- Fetch all products for that business
- Display on UI

Outputs:

Local Pulse

- Business details page with products

Error Handling:

- Business not found
- DB error

3.2.7 Contact Business (Customer)

Introduction: Customers can contact businesses quickly.

Inputs:

- business phone/email/website

Processing:

- UI provides clickable actions (call/email/open website/map)

Outputs:

- Customer can contact business using their device apps

Error Handling:

- Missing contact details → show “Not available”

3.5 Non-Functional Requirements

3.5.1 Performance

- At least 95% of product search requests should return results within 2 seconds under normal load.
- Home/Search page should load within 3 seconds on a normal internet connection.
- Search results should be paginated (default 20 results per page) to improve speed and reduce data load.
- The backend API should support at least 100 concurrent active users without crashing (basic target).

3.5.2 Reliability

- Server error rate (5xx errors) should be less than 1% of total requests per day under normal operation.
- The system should show clear error messages for failed actions (login failed, product not found, etc.).
- Database should use proper constraints/relations to avoid invalid or duplicate records.
- Daily automated backups should be enabled using Supabase (or an equivalent method).

3.5.3 Availability

- The deployed system should target 99% uptime per month (excluding planned maintenance).
- Planned maintenance should be minimal and ideally under 2 hours per month.
- If Supabase or any service is temporarily down, the system should show a friendly retry message instead of crashing.

3.5.4 Security

- All communication between frontend and backend should use HTTPS in production.
- Only logged-in business owners can create/update business profiles and manage product listings.
- Role-based access must be enforced: business owners can manage only their own business and products.
- Passwords must not be stored in plain text (use Supabase Auth or secure hashing).
- Inputs must be validated/sanitized to avoid attacks like SQL injection and XSS.
- Login and search endpoints should have basic rate limiting to reduce abuse.

3.5.5 Maintainability

- Code should be modular with a clear folder structure and reusable components.
- TypeScript types/interfaces should be used for API request/response data.
- Follow consistent coding style using tools like ESLint/Prettier.
- Unit tests should cover main features (target at least 60% for core services).
- APIs should be documented (README, Swagger, or Postman collection).

3.5.6 Portability

- The frontend should work on modern browsers (Chrome, Edge, Firefox) and support mobile screens.
- The project should be deployable on common platforms (example: Vercel/Netlify for frontend and Render/Railway for backend).
- Environment variables should be used for configuration (DB keys, API URLs) instead of hard-coded values.

3.7 Design Constraints

- Frontend must be built using React + TypeScript
- Backend must be Node.js + Express
- Database must be Supabase PostgreSQL
- Deployment must support:
 - React build hosting
 - Node API hosting
 - Supabase DB connection via environment variables

3.9 Other Requirements

- Product image upload limit (example: max 2MB)
- Basic audit info (created_at, updated_at)
- Optional “featured businesses/products” section on home page
- Basic search analytics (optional)

4. Analysis Models

Level 0 (Context Diagram)

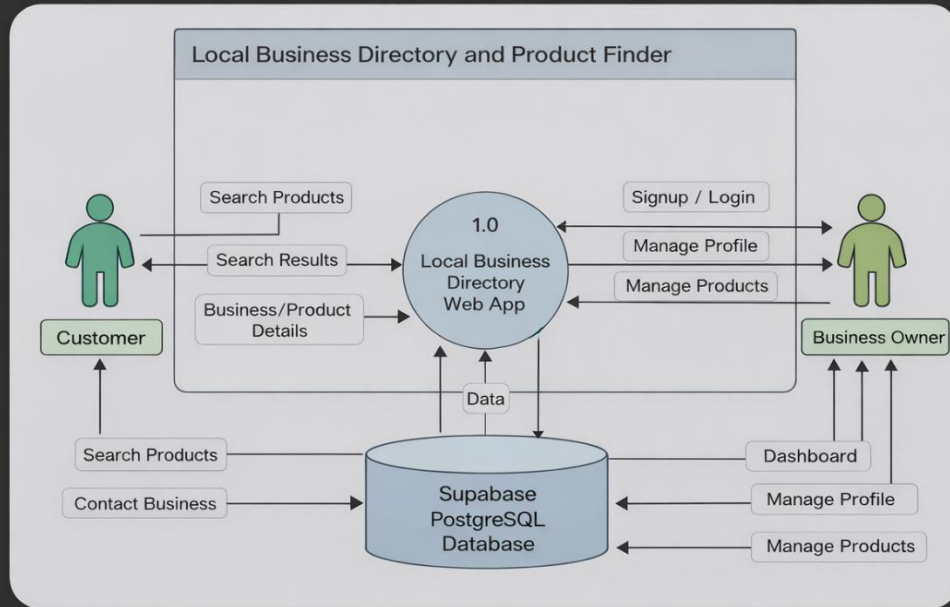
Customer / Business Owner → Local Directory & Finder System → Supabase PostgreSQL
System returns results/pages back to users.

Level 1 (Main Processes)

1. Search Products
 - Customer enters query + filters
 - System queries DB
 - Results returned
2. Manage Business Profile
 - Owner submits profile form
 - System validates and stores in DB
3. Manage Products
 - Owner adds/edits/deletes product
 - System updates DB
 - Updated product list shown

4.1 Data Flow Diagrams (DFD)

4.1 Data Flow Diagrams (DFD)



5. GITHUB LINK

<https://github.com/Anand-1105/LocalPulse>

6. DEPLOYED LINK

: local-pulse-five.vercel.app

A. Appendices

The appendices provide additional supporting information related to the Local Business Directory and Product Finder system. The information included in these appendices is not considered part of the core functional requirements of the SRS, but is provided to help understand the system design, tools, and assumptions used during development.

A.1 Appendix 1 - Technology Stack and Tools

This appendix describes the technologies and tools used to develop the system.

- **Frontend:** React with TypeScript is used to build a responsive and user-friendly interface for customers and business owners.
- **Backend:** Node.js with Express is used to implement REST APIs and handle business logic.
- **Database:** Supabase (PostgreSQL) is used to store user accounts, business profiles, and product information.
- **Authentication:** Supabase Auth is used for secure login and role-based access control.
- **Development Tools:** Visual Studio Code, Git, and GitHub/GitLab are used for development and version control.

A.2 Appendix 2 - Assumptions and Future Enhancements

Assumptions

- Users have access to the internet and a modern web browser.
- Business owners provide accurate and up-to-date product and business information.
- System availability depends on third-party services such as hosting platforms and Supabase.

Future Enhancements

- Online ordering and payment support.
- Mobile application version of the system.